Mini Banking System using Layered Architecture (Service + Repository)

1. Introduction

The Mini Banking System is a desktop-based application developed using Java that simulates basic banking operations such as account creation, deposit, withdrawal, balance enquiry, and authentication. The project follows a layered architecture pattern, specifically separating concerns into Service and Repository layers, which improves maintainability, scalability, and testability.

This system is designed as a learning project to demonstrate concepts such as object-oriented programming, database connectivity, and software architecture design principles.

2. Objectives

- To develop a banking system that performs core banking operations
- To implement layered architecture (Service + Repository pattern)
- To separate business logic from data access logic
- To demonstrate the use of JDBC for database interaction
- To build a modular and maintainable Java application

3. Technologies Used

- Programming Language: Java
- Database: H2 Database
- IDE: IntelliJ IDEA / VS Code
- Libraries:
    - H2 Database Engine
    - JCalendar (for date handling)

4. System Architecture

The project follows a layered architecture consisting of:

4.1 Presentation Layer

- Handles user interaction (UI forms such as Login, Signup, Transactions)

-
- Collects input and displays output

4.2 Service Layer

- Contains business logic
- Validates user inputs
- Processes operations like deposit, withdrawal, etc.

4.3 Repository Layer

- Handles database operations
- Executes SQL queries
- Communicates with the database using JDBC

4.4 Database Layer

- Stores user account details and transaction records
- Implemented using H2 database

5. Project Structure

The project maintains the following structure:

```
Bank-Management-System-master/
|
 ├── src/
|   └── bank/management/system/
|       − BalanceEnquiry.java
|      -- Conn.java
|       — Deposit.java
|      — FastCash.java
|       — Login.java
|       — main_Class.java
|       — mini.java
|       — Pin.java
|       — SchemaInit.java
```

```
|      — Signup.java
|      — Signup2.java
|      — Signup3.java
|      └— Withdrawal.java
|
— libs/
|  — h2-2.4.240.jar
|  — jcalendar-tz-1.3.3-4.jar
|
— icon/
|  — atm2.png
|  — backbg.png
|  — bank.png
|  — card.png
|  └— provider.png
|
— docs/
— report/
— slides/
— README.md
— bankSystem.mv.db
— bankSystem.trace.db
```

6. Module Description

6.1 Login Module

- Authenticates users using account number and PIN
- Redirects to the main menu upon successful login

6.2 Signup Module

- Multi-step registration process

- Collects personal and account details

6.3 Deposit Module

- Allows users to deposit money into their account
- Updates database records

6.4 Withdrawal Module

- Enables users to withdraw money
- Checks for sufficient balance before processing

6.5 Fast Cash Module

- Provides predefined withdrawal amounts for quick transactions

6.6 Balance Enquiry Module

- Displays current account balance

6.7 PIN Change Module

- Allows users to change their PIN securely

6.8 Mini Statement Module

- Shows recent transaction history

7. Database Design

The system uses an H2 database with tables such as:

- **Users Table**: Stores user details
- **Accounts Table**: Stores account information
- **Transactions Table**: Stores transaction records

Database connectivity is handled through the Conn.java class.

8. Key Features

- User authentication system
- Secure transaction handling
- Real-time balance updates
- Simple and user-friendly interface
- Modular and scalable architecture

9. Implementation of Object-Oriented Programming (OOP) and Challenges Faced

The Mini Banking System is developed using Object-Oriented Programming principles to ensure modularity, reusability, and maintainability. The project applies key OOP concepts such as encapsulation, abstraction, inheritance, and polymorphism. During implementation, several challenges were encountered while applying these concepts effectively.

13.1 Encapsulation

Encapsulation is used by organizing data and related methods within classes such as Login, Deposit, and Withdrawal.

- Sensitive data like PIN and account details are handled within classes
- Access is controlled through methods

Challenge:
Maintaining proper data flow between different classes without breaking encapsulation was difficult, especially when multiple screens needed the same data.

13.2 Abstraction

Abstraction is implemented by hiding complex database operations inside the Conn class.

- Users interact only with the interface
- SQL queries and database handling are hidden

Challenge:
Designing abstraction properly was challenging because database logic and UI logic were initially mixed, requiring restructuring of code.

13.3 Inheritance

Inheritance is used where classes extend existing classes such as JFrame to reuse GUI functionality.

Example:

Challenge:
Understanding when and where to use inheritance correctly without making the code overly dependent on parent classes.

13.4 Polymorphism

Different transaction classes (Deposit, Withdrawal, FastCash) perform similar operations in different ways, demonstrating polymorphism.

Challenge:
Applying polymorphism effectively was difficult because methods were initially implemented separately without a common interface or structure.

13.5 Overall Challenges in OOP Implementation

- Difficulty in structuring the project into proper layers (Service and Repository)
- Managing communication between classes
- Avoiding tightly coupled code
- Debugging issues across multiple interconnected classes
- Ensuring reusability without increasing complexity

```java
public class Connn {
    Connection connection;
    Statement statement;
    public Connn(){
        try{
            // Use an embedded DB so the project runs without requiring MySQL locally.
            // Data is stored in the local folder as ./bankSystem.*.
            connection = DriverManager.getConnection(url: "jdbc:h2:./bankSystem;AUTO_SERVER=TRUE");
            statement = connection.createStatement();
            initSchema();
        }catch (Exception e){
            e.printStackTrace();
        }


    }

    private void initSchema() throws SQLException {
        // Table schemas are created to match the insert/select patterns used throughout the app.
        statement.executeUpdate(
                "CREATE TABLE IF NOT EXISTS login (" +
                        "formno VARCHAR(50), " +
                        "card_number VARCHAR(20), " +
                        "pin VARCHAR(10)" +
                        ")"
        );
```

- CODE TEMPLATE

```java
@Override
public void actionPerformed(ActionEvent e) {
    try {
        String amount = textField.getText();
        Date date = new Date();
        if (e.getSource()==b1){
            if (textField.getText().equals(anObject: "")){
                JOptionPane.showMessageDialog(parentComponent: null,message: "Please enter the Amount you want to Deposit");
            }else {
                Connn c = new Connn();
                c.statement.executeUpdate("insert into bank values('"+pin+"', '"+date+"','Deposit', '"+amount+"')");
                JOptionPane.showMessageDialog(parentComponent: null,"Rs. "+amount+" Deposited Successfully");
                setVisible(b: false);
                new main_Class(pin);
            }
        }else if (e.getSource()==b2){
            setVisible(b: false);
            new main_Class(pin);
        }
    }catch (Exception E){
        E.printStackTrace();
    }

}
```

CODE TEMPLATE

10. Limitations

- Basic UI design
- Limited security features
- Not suitable for real-world banking use without enhancements

11. Future Enhancements

- Integration with advanced databases like MySQL
- Improved UI using JavaFX
- Implementation of encryption for security
- Adding online banking features
- Mobile application support

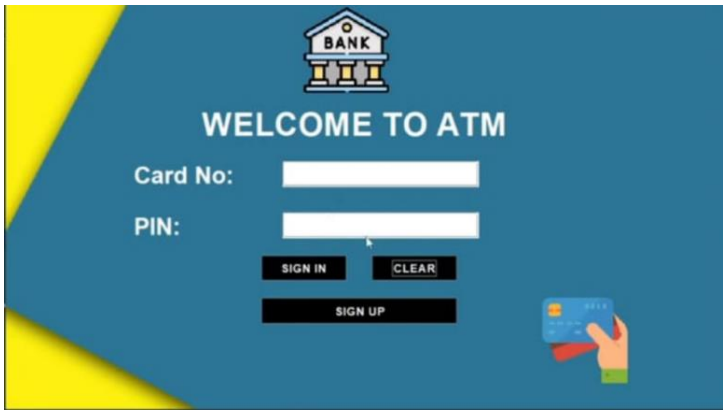
12. Graphical User Interface (GUI)

The system uses Java Swing for GUI design. The interface includes:

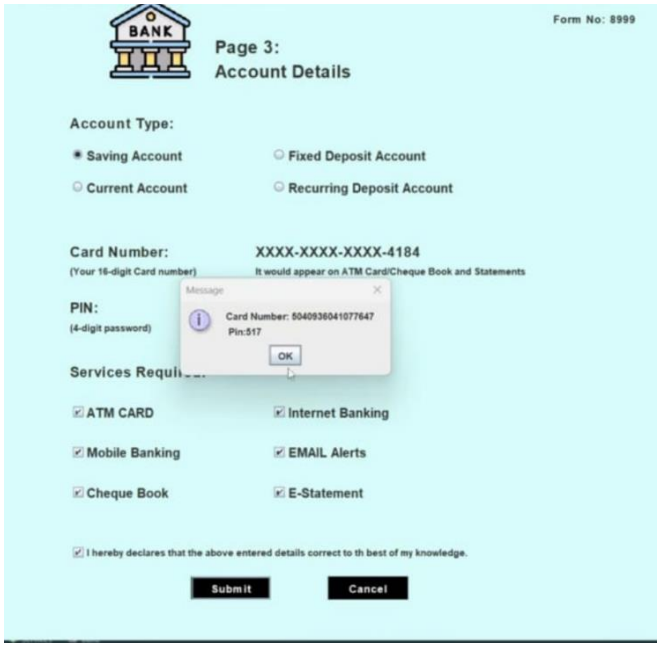
- Login screen
- Signup forms
- Transaction menu
- Deposit and withdrawal screens

Images and icons are used from the icon folder to enhance visual appearance.

GUI PICTURE:



CHANGE YOUR PIN
New PIN:
Re-Enter New PIN:
CHANGE
BACK
1
2
3
4
5
6
7
8
9
0
00
Confirm



13.conclusion

The Mini Banking System project successfully demonstrates the implementation of a basic banking application using Java and a layered architecture approach. By separating the system into Service and Repository layers, the project achieves better organization, modularity, and maintainability.

Through this project, core banking functionalities such as account creation, deposit, withdrawal, balance enquiry, and transaction handling were effectively implemented. The integration of a relational database using SQL and JDBC ensured efficient data storage and retrieval.

Additionally, the project provided practical exposure to Object-Oriented Programming concepts such as encapsulation, abstraction, inheritance, and polymorphism. It also helped in understanding the importance of structured coding practices and clean architecture design.

Despite some challenges in database connectivity, GUI management, and code structuring, the project was completed successfully and serves as a strong foundation for building more advanced applications in the future.

Overall, this project enhanced both technical knowledge and problem-solving skills, making it a valuable learning experience in software development.